

Fire Age Main-Eterra Dungeoneers

GDD #3: Building an Agent Crew

Khi-Rho Studios | Piero L. S. Folly | 3 November 2026

[F.A.M-E.D.] is a co-op exploration, dungeon and deck-building game for 1 to 4 players. This report shows the **[agent crew]** I built for it: the studio roles, how the agents pass work to each other, the game data they make, the checks they run and a playable **[dungeon combat]** prototype. It connects nine test builds and my final **[gameplay feedback]** to create an optimized studio crew idea for the production workflow I plan to use.

1. THE AGENT CREW

Required deliverable	In this document
Crew code	The agent instructions, the checks and the **[runner]** script for a six-card test. I can share better details as needed.
Mermaid diagram	**[crew-diagram.mmd]**, also drawn in the **[README]**. This report also shows my hand sketch, the **[Test Crew 8]** chart and my studio crew chart.
[README]	Names the game, the output, the crew roles and the folder map, and says what the files can show.
Game-ready output	A playable dungeon combat prototype from **[Test Crew 8]**, with its card and rule data. I can show it on request. I have not imported it into **[Unreal Engine]** yet, but tested in Web games and **[Godot]** successfully.

Nine test crews

For this report I ran nine **[test crews]**. Each one used its own workflow **[chart]** and built the same **[Combat]** demo (competing with one another for a grade). Each **[crew]** is numbered in build order, and the Legend names its **[chart]**. **[Test Crew 8]** was the winner and its **[chart]** puts a **[format check]** before every **[other check]**.

What ran live and what is a replay

I kept the **[records]** of four separate agent calls from **[Crew 8]**. They are not in the **[repository]**; I can share them on request. The **[Lead Designer]** (**[Claude]**) started those agents with the Agent tool in **[Claude Code]**. The replay in the **[repository]** makes no new model calls. The six-card **[runner]** script in the **[repository]** has no model connection, and I have not approved any of its output for release.

2. THE GAME AND THE OUTPUT

Up to four players each control one character and fight enemies together on a dungeon grid. Cards give the actions and the resources. The goal of this course is to fill the full **[Combat]** loop. If time allows, the **[Dungeon Crawl]** is next, and after that **[Landscape]** exploration (**[GDD]**, section 1).

Loop	What the crew must support	Status
[Combat]	Card piles, movement, attack, defense, card choices, enemy turns, victory, defeat and restart.	Course goal. All nine test builds play a dungeon boss fight and follow the **[rulebook]** combat rules.
[Dungeon Crawl]	Encounters, tiles that stay on the board, and the move from exploring into combat.	Next in production, if time allows. Test builds start directly in the fight at this stage.
[Landscape]	Exploration, events and the route between places.	After the **[Dungeon Crawl]**, if time allows. No building plan for it yet.

A small production task (test-drive)

The **[Card Crew]** (**[Card Data]**, **[Rule Audit]**, **[Print Match]** deployed by the **[Art Director]** as three **[Codex]** agents; **[validate.py]** checked their work) works on six cards. I designed these cards and gave them their IDs from the **[Component Studio]**'s CSV file when I made the tabletop game in **[TTS]**. The crew reads them from the CSV file and my printed art PDFs.

ID	Card	Status
SK1_01	**[Knockout!]**	**[Rule Audit]** flag: one data value is missing. Print read. Used in 4 **[test crews]**.
SK1_02	**[Double Strike]**	**[Rule Audit]** flag: the text says +4, the data says 3. Print read. Used in all 9 **[test crews]**.
SK1_03	**[Mental Push]**	**[Rule Audit]** flag: reward symbols open; round or turn unclear. Print not readable.
SK1_04	**[Salvage Resources]**	**[Rule Audit]** flag: the text names the R.F.D. pile, the data names discard. Print read.
SK3_01	**[Stand as One]**	**[Rule Audit]** pass. Print read. Used in all 9 **[test crews]**.
SK4_01	**[Edan's Leap]**	**[Rule Audit]** pass. Print read.

What is usable now

[Test Crew 8] made the **[best playable]** web version of the **[combat]** test, its **[card data]**, a **[rule audit]**, the **[checks]** and a **[record]** of each step. The six-card **[runner]** repeats the same checks offline. Its end product is **[DT_Cards.csv]**: a spreadsheet file with one row per card, which the engine loads as its card table. The **[runner]** holds this file back when it cannot confirm the printed card, so unclear card text never becomes game data.

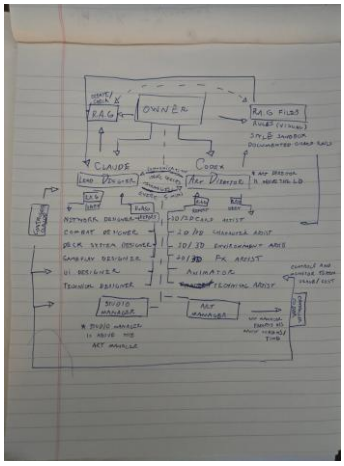
3. THE STUDIO SETUP

My original sketch split design ([Claude]) and art leadership ([Codex]), with approved source material, specialists and cost control. The [Studio] now adds a [mailbox] for each role in it (environment folder for each sole agent), dated [handoff] files and small scripts to relay messages when other methods fail. [Hook] scripts run on their own at chat events: one checks each reply against my writing rules, while another plays a unique chat-type flavored sound when a reply lands. [Hook] examples in the background: [Watcher] checks each mailbox every 60 seconds, and [Budget Guard] checks usage every 5 minutes, and [Controller] reports old work, mailbox gaps and budgets.

Role	Responsibility	Boundary
[User] (studio owner)	Sets the game goal and mood, the idea direction, the priorities and the final choices. Participates in every endeavor possible.	The only one that can use [/clear] or delete anything (all agent deletions move to a [_limbo] folder instead). It has no boundaries at any studio work. It can interfere and modify the workflows and pipelines at will.
[Studio Director], [Codex]	Coordinates the studio at large by conversing ideas and coming to conclusions, reviewing agents task results or monitoring them directly if necessary. When the [User] is away, it can have power of user-only voice for a set time making decisions as long as it doesn't break Boundaries, i.e. deleting data without the [User] direct and active consent.	When making choices: first it answers from the [RAG files], then when gaps, errors, or inconclusive events happen, it refers to the [User] directly for guidance. It labels and log with the [agent decision-agent name] each answer it gives for when the [User] is absent, but never turns an [unknown] into a fact without the [User] consent or [RAG files] without flagging the same unique events for review later.
[Art Director], [Codex]	Directs presentation, sources and independent checks. Leads the aesthetics, visual organization and visual language of the project.	Keeps the original evidence and states what it checked. Reviews the work of the [Lead Designer], and the [Lead Designer] reviews its work. Follows the [decision rule] of the [Studio Director].
Studio Help Spawner, Codex	Starts scoped Codex helper agents when a role asks. Each one gets a goal, its files and their owner, a check and a reply route.	Starts only the agents a role asks for. Sends each result to the mailbox of that role. Follows the decision rule of the Studio Director.
[Studio Manager], [Claude]	Tracks work, model settings, usage, mailbox coverage, open items and [handoffs].	Reports and alerts only. Its scripts alert at set usage levels, and each chat stops itself on an alert. The crew [runner] has its own hard cap: 10 calls and 400,000 tokens. Follows the [decision rule] of the [Studio Director].
[Lead Designer], [Claude]	Envisions and designs the game and the crews. Sends tasks and joins the results.	Design only. Anything to build goes through the [Lead Engineer]. Discusses each design choice with the [Art Director]. Follows the [decision rule] of the [Studio Director].
[Lead Engineer], [Claude]	Builds and runs what the studio and the project need: environment, engine, scripts, hooks and its own subagents.	Everything functional and structural. Follows the design of the [Lead Designer] and sets no design. Follows the [decision rule] of the [Studio Director].
Lead Environment Engineer, Claude	Sets up and keeps the studio environment: the watchdog, the mailboxes and the Claude and Codex setup.	Environment only. Builds no game and sets no design. Follows the decision rule of the Studio Director.
[Lead Web Designer], [Claude]	Maintains navigation, the published pages and the artifacts.	Keeps every page and its history. Builds the [artifact] and checks every fact on it and in every published output. Checks that the [Studio Manager] makes each agent update the [Dev Log] in its own section. Follows the [decision rule] of the [Studio Director].

What every task names

Each task names its game and folder, approved inputs, output format, test, budget and stop condition. The agent that makes a file owns that file. A checker reports a fault, and the owner of the file fixes it. The [Studio Manager] tracks the [handoff], and I settle each real design choice. Every agent uses the [decision rule]: [RAG files] first, my guidance on gaps, an [agent decision-agent name] on each choice made without me, and no [unknown] turned into a fact.



My original design (image on the left)

My pen-and-paper sketch from October 27th. I sit at the top as the [Owner]. [Claude]'s [Lead Designer] and [Codex]'s [Art Director] sit below me and talk to each other every 5 minutes about ideas, checks and strategies. In the sketch the [Art Director] ranks above the [Lead Designer].

Each lead runs its own specialists: network, combat, deck, gameplay, UI and technical designers on the [Claude] side; card, character, environment and effects artists, an animator and a technical artist on the [Codex] side. A [Studio Manager] and an Art Manager report tokens and time, and a controller on each side watches token usage and cost.

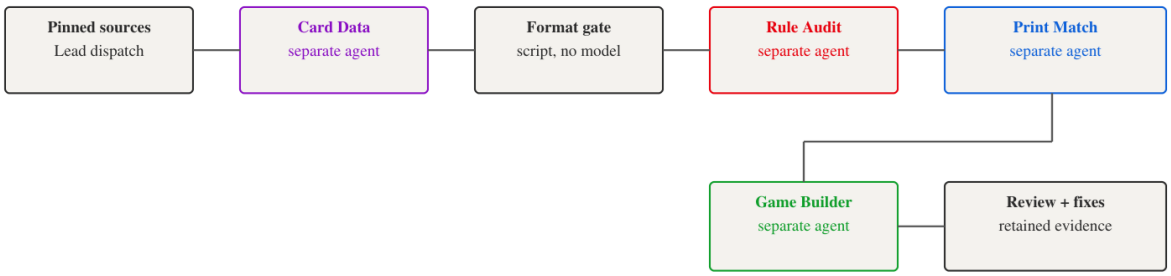
How it changed

The sketch shows the whole studio. Section 4 shows one task inside it: my sources become checked card data, then a playable build.

The [RAG files] are the source documents each agent receives: the [GDD], the [rulebook], the data CSV and the art PDFs. They have no search index yet. The style sandbox is not built.

4. ONE WORKING CREW: TEST CREW 8

[Test Crew 8] is the clearest small case. The [Lead Designer] called a separate agent for each of [Card Data], [Rule Audit], [Print Match] and [Game Builder]. I kept the call ID, model, inputs and time of each call; I can share them on request.

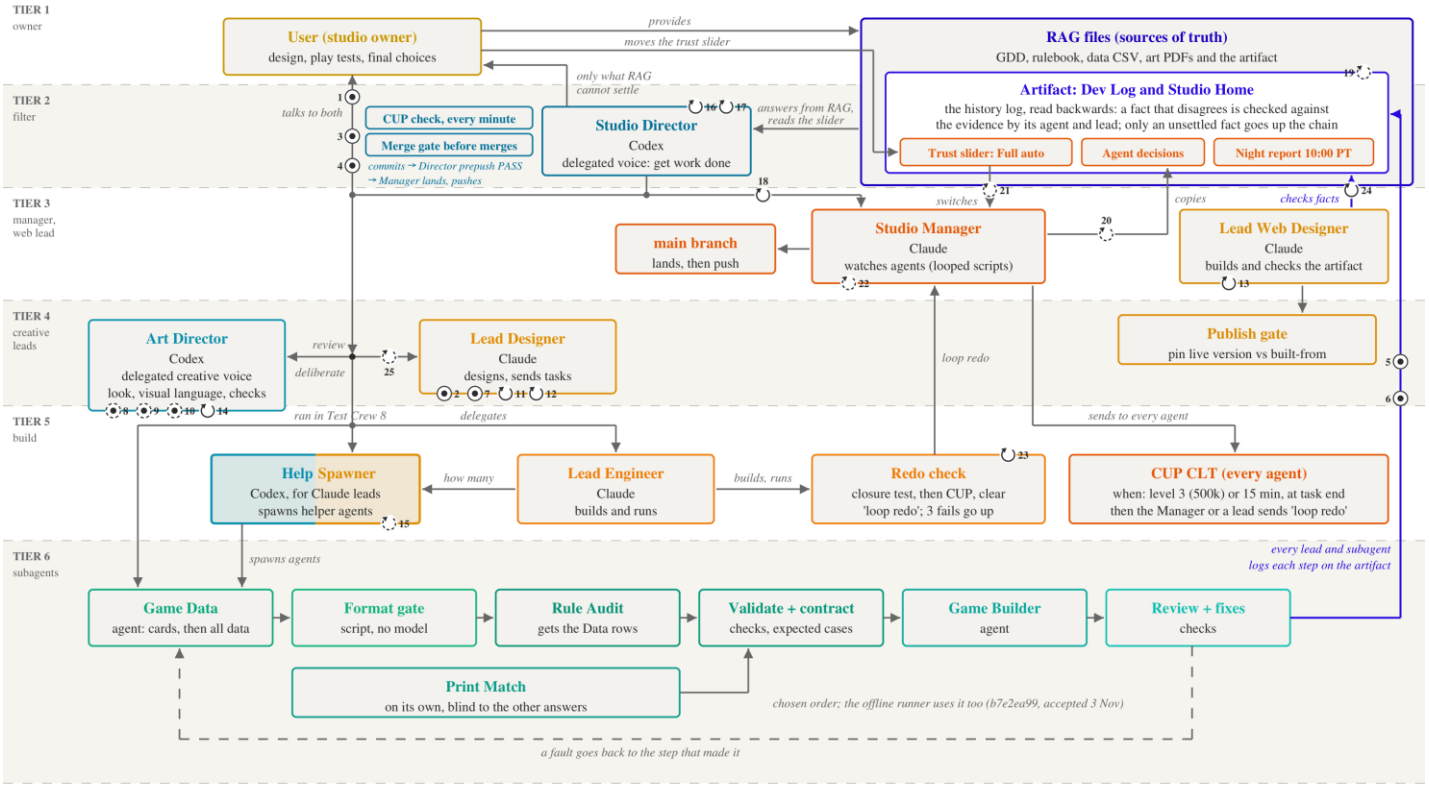


Step	Input to output	Why it exists
[Card Data]	Approved sources -> combat-data.json	Makes structured card rows before anyone writes game code.
[Format gate]	Rows -> pass, or a list of format faults	A script finds a wrong shape at no model cost.
[Rule Audit]	Rows and sources -> audit.json	A second agent checks the meaning of each rule and lists open decisions.
[Print Match]	Card images -> print notes	Compares the printed card with the data. In the first pass it received art-only images; I kept the later correction.
[Game Builder]	Accepted data, audit, print notes and my rulings -> playable build	Connects the card data to a working combat loop.
Review and fixes	Built files -> faults, fixes and checks	Finds and fixes what the first build got wrong.

[Test Crew 8] needed three fixes: the last card falling, a buff that ended at the wrong time, and the use of defense. I kept those faults in the [record].

My studio crew: plan moving forward (partly tested)

The [Test Crew 8] chart shows the steps but not my studio roles. The chart below places those tested steps inside the studio from section 3. Solid lines ran in [Test Crew 8]. Dashed lines are my plan and are not tested yet: in the plan, the [Lead Engineer] starts the agents and the [Lead Designer] only designs. In the plan, the [Card Data] step grows into [Game Data]: the cards and all other game data.



Studio loop (runs around every task)

Flow: the user talks to the Art Director and the Lead Designer. They review and deliberate, then send each job to the Lead Engineer, the Help Spawner or the subagents.
Filter: the Studio Director answers from the RAG files and sends up only what they cannot settle. The trust slider is on Full auto (user, 04:16 PT).
Away: the Art Director speaks for creative choices and the Studio Director gets things clear and done. Tiers: the bands show the studio order.
Facts: every lead and subagent logs each step on the artifact; the Lead Web Designer checks each fact. A wrong fact goes to its agent and lead, then the Lead Designer or the Art Director, then the Studio Director, then the user. Merge gate: no merge before the Director names the newest side.
CUP CLT: at a task end after context level 3 (500k) or 15 minutes, each agent commits its own paths, updates the pages, sends a closure packet, clears, waits for 'loop redo'.
Push: local commits, then the Director's prepush PASS; the Studio Manager lands approved merges on main and runs the push. Commit subjects start with the role name.
Worktrees: a chat cannot write outside its own worktree. After a landing, each chat works from main. Publish: pin the live version against the built-from version.
Marks: each number names a source or config file. Solid = the control runs and its owner confirmed it; dashed = prepared, off, open defect, or not confirmed.
Limits: every Claude chat stops at 85% of the 5-hour limit. Night report: on Home at 10:00 PT, notes from the Studio Manager. /clear only after a handoff file. _limbo: nothing is deleted.

Hook: runs on an event in a chat. **Script: runs on a timer or on demand.** **Dashed: prepared, off, open defect, or not confirmed by its owner.**

1 rules_turn.py: each prompt, all Claude chats. Adds the clock and the known terms.
2 rules_guard.py: each tool call, all Claude chats. Checks the call against the rules.
3 rules_check.py: each reply end, all Claude chats. Checks the writing rules, logs terms.
4 play-done-sound.sh: each reply end, studio Claude chats. Plays the done sound.
5 level_hook.py: each reply end, studio Claude chats. Logs the context level.
6 r15.py: each artifact write, studio Claude chats. Checks the write.
7 inbox-watch-hook.json: chat start, studio Claude chats. Starts the chat's own inbox watcher.
8 session-start.mjs: Art Director chat start. Keeps one inbox watcher on. Trust unknown.
9 lifecycle.mjs: Art Director prompt, tool and stop. Busy and idle marker. Trust unknown.
10 codex-art-director-notify.py: Art Director turn end. Done sound, not tested yet.
11 inbox-watch.sh ROLE: every 60 s. Wakes each studio Claude chat on its own inbox.
12 budget_guard.py: every 5 min, inside 11. Usage meter alert.
13 worker-inbox-watch.sh: every 60 s. Inbox watcher of each worktree worker.
14 codex-inbox-watch: every 60 s. Wakes the Art Director on a new message.
15 Help Spawner watcher: every 60 s. Wakes the Help Spawner. Reported, not owner-confirmed.
16 Director main loop: every 1 min. Inbox, CUP checks, merge verdicts.
17 Director check-in: on each Director turn; owner-verified (inbox_health.py, 04:42, 04:46 PT). Not always on.
18 Director inbox health: every 1 min (Windows task). Alerts the Manager after 12 to 15 min.
19 burn_meter.py: OPEN defect (Director 136). Usage burn meter, not healthy.
20 devlog_sync.py, cup_lite.py: every 10 min. Copies events and decisions. Live writes paused.
21 apply_switches.py: with the sync. Applies the Home Scripts switches. Live writes paused.
22 Studio watchdog: at logon. Chat health; alarms OFF (owner, 1 Nov).
23 redo_check.py: at each redo. Closure test and decision gate. decision_check.py: unused.
24 check_artifact.py, watch.sh: on demand. Fact check of the artifact.
25 studio-bridge: on demand only. One-shot channel from Codex to Claude.

5. THE CODE AND THE CARD CREW

The **[test crews]** ran through the Agent tool in **[Claude Code]**. Their checks and game code are plain source files in each crew folder.

The **[card crew]** code is plain Python. **[run_crew.py]** packs the approved sources, gives each step its own packet, checks each answer and keeps the first print read unchanged. It has no model connection of its own; it replays saved answers. The **[public repository]** holds the code and one saved replay record. My private sources stay out, so the public copy cannot rerun the full chain.

Part	What it does	Limit
Fixed sources	Checks the allowed file list and the file fingerprints before each step.	A matching fingerprint shows that a file did not change. It does not show that its rule is right.
Three separate tasks	**[Card Data]** makes the rows, **[Rule Audit]** checks them, and **[Print Match]** sees only names, IDs and card images.	Section 13 shows the one run with real agents.
Answer shape	Each step returns a fixed JSON shape. A wrong shape retries that step only.	Four tries per step. The 10-call cap wins.
Caps	Stops a run at 10 calls or 400,000 tokens.	Covers this **[runner]** only, not every studio chat.
Stops	A source mismatch, a usage stop or a used-up retry stops the run and keeps the reason.	No paid service, no new account, no automatic fallback.
Release	When a printed card cannot be confirmed, the **[runner]** keeps its notes and holds back **[DT_Cards.csv]**.	My review is still required.

The run order, step by step

```
make_new_run_folder_and_lock()                          # live mode and a skipped stage stop first
check_sources()                                         # again in every packet, before every try, and in validate
on their own, in any order:
    rows = ask("Card Data", data_packet)
    print_read = ask("Print Match", card_images_ids_names)# first raw read kept, never replaced
    check_format(rows), check_format(print_read)        # only a format fault tries again
audit = ask("Rule Audit", audit_packet, rows)           # gets the rows, never the print read
check_format(audit)
result = validate(rows, audit, print_read)
write_validation_and_qa_report(result)                  # no DT_Cards.csv before a valid result and the user's release
stop_for_user_if_not_valid(result)                      # the release always waits for the user
save_run_log()                                          # input and output hashes, calls, tokens, status
```

This is the order the crew runs in from now on, as in the 1 November run: **[Card Data]** and **[Print Match]** on their own, then **[Rule Audit]** with the [Card Data] rows. Since 2 November the offline **[runner]** replays the same order (commit b7e2ea99; the [Art Director] accepted it on 3 November). [Print Match] stays blind to the other answers. The **[runner]** caps a run at 4 tries per stage and 10 calls.

6. OUTPUTS, CHECKS AND FAILURE

For me, done means an output plus proof for it. A model that says "pass" gives one claim. A script check, a second agent that finds a fault and my own play test are separate proofs.

Proof	What it shows	What it does not show
Four separate agent calls	**[Test Crew 8]** ran each step as its own agent call, with its inputs and models on **[records]**.	That every chart box ran, or every rulebook rule.
Game and data files	The agents made a playable **[F.A.M-E.D.]** prototype and its card data.	Final balance or a finished product.
[Test Crew 8] fixes	Fault reports and later checks cover the falling card, the buff timing and the defense.	A full proof of the game.
My **[gameplay feedback]**	I played the nine builds and wrote what I liked, what confused me and what broke.	A formal usability study.
[card crew] tests	The fix run passed 34 tests, with 0 failures and 0 new model calls. A later full run passed 65 tests and skipped 1.	A new test run today.
Replay result	The **[runner]** code and its replay log are in the **[repository]**.	A print check on paper, a new model run or an **[Unreal]** import.

What the card data must keep

The card ID and the display name stay separate. The printed text stays beside the values read from it. Timing, the pile after use, tags, count and source are checked. A field I have not decided stays open. A printed exception is never replaced by a rule the model prefers.

A safe stop is an output

When a card cannot be read, a source is missing or my approval is open, the crew records the gap and holds the release. It keeps its answer and the reason. It does not open a new account, call a paid service or change an earlier result. It tags the stop with its **[agent decision-agent name]** and flags the **[unknown]** for my review.

7. NINE PROTOTYPES, DIFFERENT VARIABLES

Nine builds played the same dungeon boss fight: two from **[Codex]** and seven from **[Claude]**. The plan of four **[Claude]** builds against four **[Codex]** builds did not finish.

Build	Builder	Engine	Crew method	Main finding
[Test Crew 1]	**[Claude]**	Web	Sequential baseline	Readable log and tabs; autoplay did not turn off.
[Test Crew 2]	**[Codex]**	Web (canvas)	Chart 3, adapted	Useful trace; weak visual guidance.
[Test Crew 3]	**[Claude]**	**[Godot]**	Joined data and code	Fewer extra clicks; space bar problem.
[Test Crew 4]	**[Claude]**	Web (Phaser)	Separate build lane	Strong styling; too many prompts.
[Test Crew 5]	**[Codex]**	Web (canvas)	Chart 6, adapted	Least liked; do not follow this direction.
[Test Crew 6]	**[Claude]**	Web (HTML and SVG)	Contract and expected cases	Smooth board and card flow; unclear diagonal moves.
[Test Crew 7]	**[Claude]**	Web (Three.js, 3D)	Proof chain	Simple 3D and camera control.
[Test Crew 8]	**[Claude]**	Web (Phaser)	Format first	Animation and short, clear feedback.
[Test Crew 9]	**[Claude]**	Web	Sealed-check chain with lanes	Helpful controls; split my attention.

What changed between crews

The engine, the agent count, the task split, the checks, the retry routes, the art and the controls all changed from build to build. Some builders did the work of several chart roles. Each crew folder keeps the planned **[chart]** beside the record of what actually ran.

Limits of the comparison

The set shows interface and workflow ideas. Many parts changed at once, so the set cannot prove which chart made the better game. The next test uses the chosen features and the same test cases for each crew. I have not run that test yet.

8. MY FEEDBACK AND THE STUDIO DIRECTOR'S READING

I gave written **[gameplay feedback]**, not grades. The **[Studio Director]** turned my words into the scores below. The scores are not averaged into a winner.

Build	Score	My words	Keep or fix	Link
[Test Crew 1]	8/10	"the Combat log font format + colors make the game very easy to follow."	Readable log and tabs; autoplay did not turn off.	Game
[Test Crew 2]	4/10	"This is the weakest game that I played so far."	Useful trace; weak visual guidance.	Game
[Test Crew 3]	8.5/10	"My two most favorite so far were the Godot, the Godot one, and right now this one, the Group 10"	Fewer extra clicks; space bar problem.	Game
[Test Crew 4]	7.5/10	"the strongest thing here is the aesthetics and the amount of icons and explanation, but that can be overwhelming too."	Strong styling; too many prompts.	Game
[Test Crew 5]	3/10	"I think this one's the one that I like the least, the number six, group six."	Least liked; do not follow this direction.	Game
[Test Crew 6]	9/10	"This one has actually been the highest grade so far."	Smooth board and card flow; unclear diagonal moves.	Game
[Test Crew 7]	7/10	"I feel like the most outstanding thing is that you can rotate the camera on this other one in the localhost"	Simple 3D and camera control.	Game
[Test Crew 8]	9/10	"But this game has the best aesthetics in terms of interaction, animation, how being concise and in a unique language."	Animation and short, clear feedback.	Game
[Test Crew 9]	6/10	"It definitely looks like a game made for an engineer, not for a gamer."	Helpful controls; split my attention.	Game

My three preferred builds

My three preferred builds were **[Test Crew 3]** (**[Godot]**), **[Test Crew 6]** (smooth 2.5D board) and **[Test Crew 8]** (pixel art). I want the fewer extra clicks of 3, the board and card flow of 6, and the animation with short guidance of 8. **[Test Crew 1]** adds the log format I liked.

Faults stay beside praise

Faults I reported: autoplay did not turn off in 1, the space bar reset the game in 3, the log did not scroll in 8, diagonal moves were unclear in 6, and the **[Test Crew 9]** split my attention. The **[Art Director]** checked these rows against my notes.

9. SELECTED DESIGN AND CREW

The next design keeps the board and the current action in front. Extra information explains the current choice and does not compete with it.

Take from	Use it for	Test question
[Test Crew 6]: board focus and card hover	Keep the eye near the action and explain a card before the player commits it.	Can a new player find the active hero, a legal target and the card result?
[Test Crew 8]: animation and event guidance	Show what changed with short motion, dice feedback and central prompts.	Does the feedback explain the action without hiding the next choice?
[Test Crew 3]: fewer clicks	Do a second step for the player only when no other choice is possible.	Does the automation stop when a player can respond?
[Test Crew 1]: readable log	Use tabs, clear type, colour and a history that scrolls.	Can the player find who acted, what happened and why?
[Test Crew 7]: camera and simple screen	Keep the camera control optional and show only current information.	Can the player return to a useful view without losing the game state?
[Test Crew 2], **[Test Crew 4]** and the **[Test Crew 9]**: inspection tools	Keep the trace, the rules, the card details and the accessibility options behind clear controls.	Can the player inspect detail while the board stays in focus?

Crew choice

The next crew reads the fixed sources, runs a **[format check]** early, checks rules and print with separate agents, writes a contract with expected test cases, builds, and then gets a separate review. Rules and look stay separate. Each agent follows the **[decision rule]**.

Combat first

First, one clear **[Combat]** loop with the right card choices and endings that work. Next, the **[Dungeon Crawl]** encounters and the tiles that stay on the board. Then the **[Landscape]**. The card IDs and the agreed event formats carry the data forward. Each phase still needs its own sources and tests.

Why these three, and why this chart

Preferred build	Crew records kept	Why I combine them
[Test Crew 3]: [Godot]; joined data and code	Checked rows and expected cases go to the builder.	Connects rule data to the code and keeps the fewer clicks.
[Test Crew 6]: smooth board; contract and expected cases	A contract of allowed values and hand-worked cases guide the build.	Makes the accepted rules and the expected results clear, and keeps the board and card hover.
[Test Crew 8]: pixel art; format first	A **[format check]** comes before the rule, print and build work. A separate review found faults after the first pass.	Stops shape faults early, then asks for a separate check and a recheck. Keeps the animation and the event guidance.

My taste picks the interface. The crew records pick the useful process steps. The **[Test Crew 8]** chart in section 4 is the clearest four-agent case that really ran.

The combined flow

Fixed sources → **[Card Data]** and **[Print Match]**, each on its own → **[format check]** → **[Rule Audit]** with the Card Data rows → contract and expected cases → **[Game Builder]** → separate rules and controls review → one fix and a second check. Open rule or print questions stop the release. A visual change never changes a combat rule.

10. THE PAGES, [CUP], [/CLEAR], and [CLT]

The studio pages show the work. The **[studio repository]** and the dated records keep the proof. A useful page says what changed, when, who did it, what was checked and what is still open.

The artifact: a backwards RAG as source of truth

The **[artifact]** is the history log of the **[Studio]**: the **[Dev Log]** and the **[Studio Home]** page. It is part of the **[RAG files]**, read backwards. Every lead and every subagent logs each step on it as it works, and the **[Studio Manager]** scripts check that no agent skips its log. Between tasks, an agent checks that the history on the **[artifact]** matches what was done, because agents forget or skip things.

The **[Studio Manager]** scripts watch the agents. The **[Lead Web Designer]** builds the **[artifact]** and watches what is published: it checks every fact on the **[artifact]** and in every visible output, such as the **[repository]** and the pages. It is the second safety net, after the agents and their leads.

Each log on the **[artifact]** names the agent who wrote it, or at least its lead. When a fact looks wrong, or two logs disagree, the **[Lead Web Designer]** contacts that agent and its lead, or the **[Studio Manager]**, and they look for the truth in the logs, files, chat history, memory and **[handoffs]**, then report back. If they cannot settle it, it goes to the **[Lead Designer]** or the **[Art Director]**, who designed the work, then to the **[Studio Director]**, and last to the **[User]**. The sources of truth are three: the **[User]**, the **[RAG files]** and the **[artifact]**.

Page	Use	Limit
[Studio Home]	Links, current work, decisions and open questions.	A page label can go out of date; the source file is the record.
Crew Tests	The nine games, their charts, reports, my feedback and the comparisons.	Planned roles, actual roles and my taste stay in separate columns.
Game Dev Logs	Changes, bugs and authors for each game.	Each later entry keeps its real date.
[Dev Log]	Studio events and task summaries with dates and times.	A local log line, a saved record and a page row can differ.
Mailboxes	One role receives a task, a question, a blocker or a result.	A message in a mailbox is not yet handled. A handled message is not sent again.

CUP: commit, update, push

[CUP] means: commit the changes that belong to the task, update the logs and the **[handoff]**, then push it to Git. Private and unrelated files stay out.

A /clear starts with a handoff

Before a **[/clear]**, the chat writes a **[handoff]**: task state, open questions, file paths, costs, mistakes and running scripts. I choose when to clear. Nothing is deleted; unwanted files go to **[_limbo]**.

The Director checks each CUP

Every [CUP] ends with one packet to the [Studio Director]. The packet names the cycle, the role, the chat, the files that role owns, its commits, the page versions and the handoff. The [Director] runs 5 checks on it, once a minute: own files committed with nothing left staged, pages that cite those commits, a new handoff that names the last commit, a push readback that matches, and the clear and resume line. A missing proof is a fail, thus a readback.

The Director reviews every decision

Before merging a branch to main, the [Studio Director] reviews every outgoing commit. Example: I ordered a review of 33 local commits (e7f0370b to 7b7a3e29). The [Director] used 3 helper agents at the highest effort and held the push for 4 fixes: a burn meter lock that deleted its own folder, a message record replaced in place of kept, a commit that swept up 10 memory files of other chats, and a hooks list row that said On for a paused writer. The last 3 closed in the correction review; the burn meter fix is still open. The push runs only after the [Director] passes it and I say yes.

The merge gate

No branch merges into the main line unless reviewed by the director or explicitly appointed subagents, and no page is published over a newer live page, until the [Studio Director] looks at both sides and names the newer one. The chat that wants the merge brings one resolved commit and the list of what it kept from each side. Work from the losing side goes to [_limbo]; Agents can't delete or run delete commands without the owner order direct order.

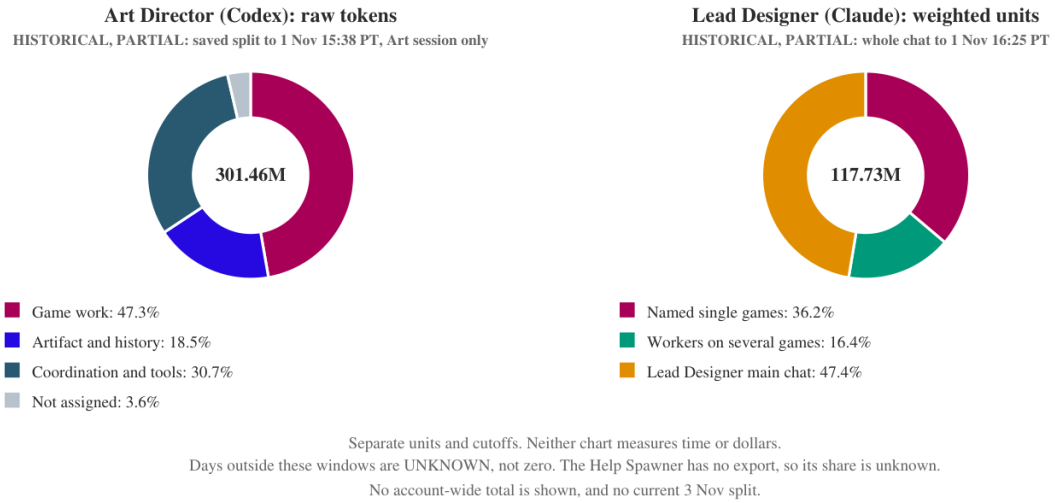
CUP CLT

When I type [CUP CLT], every agent does its [CUP], then [CLT] = clears itself [C], loops back [L] from its own handoff, and answers with one table [T]: up to 10 past items in one line each, everything done that day in two lines each, and the open items last, where I read them. Older done items move to the role's archive file, so the live handoff stays short.

11. KEEPING THE WORK UNDER BUDGET

The **[Studio Manager]** tracks usage and routes the work. Scripts do the checks that need no model. Short task packets, few retries and saved answers cut repeat calls. Each budget alert must lead to an action.

Control	What it does	Limit
[card crew] caps	Stops at 10 calls or 400,000 tokens; retries only format faults.	This **[runner]** only.
[Codex] budget reader	Reads the **[Codex]** usage and sends one warning per new level.	Alerts only; it does not stop work.
[Budget Guard]	Reads the saved **[Claude]** usage meter every 5 minutes and alerts the chats.	Alerts only; each chat must stop itself.
[Controller]	Checks old work, mailbox coverage, **[Watcher]** state, budgets and rule drift.	Status only; it is not a security wall.
Task split records	Keep the coordination work apart from the game work.	Not a price per game.
Reserve	Keep at least 10% of the **[Codex]** week. No automatic reset, account switch or paid fallback.	An account-wide meter change is not the cost of one task.



*Left: the **[Art Director]** (**[Codex]**) tokens by kind of work. Right: the **[Lead Designer]** (**[Claude]**) work units by game. Each chart has its own total and scale, so the two are never added. The left chart stops at 15:38 PT on 1 November, where the [Art Director] saved its split. The right chart covers the whole [Lead Designer] chat, to its end at 16:25 PT that day. No exact time split or cost per task exists.*

12. THE JOURNEY AND WHAT CHANGED

Period	Work and change	Lesson
26 to 28 October	Source rules, GDD revisions, the six-card scope, answer formats and source limits.	A fixed card ID and a fixed source packet work better than a large unsorted folder.
28 to 29 October	The **[card crew]** **[runner]**, its retry routes, the print limit and the failure cases were fixed.	A test with made-up data is not a real print check.
31 October	Document and code review, activity records, **[handoffs]** and the watchdog showed gaps in coordination.	Each role needs its own mailbox and a clear result.
1 November, night	The **[test crews]** made nine combat builds with different charts, engines and interfaces.	A plan on paper differs from what a crew actually runs. Crews of different sizes cannot pick a winning model.
1 November, my play test	My notes moved the focus to the board, readable feedback and fewer clicks.	A game that passes the scripts can still confuse the player.
1 November, close	Before each clear I saved the logs, counts, commits, token splits and mistakes.	Keep the reason for each fix, not only the final result.

Mistakes that made the system better

A sender filter missed real messages. A blurred screenshot gave a wrong log count. Some later records named the wrong author. A source-only check almost got reported as a live result. Each mistake led to a clearer label or a better check.

The merged mistakes list has 77 rows across six chats. The rows include incidents, blocks and game faults. Some rows repeat across chats, and an environment block is not a model mistake, so the total is not an AI error rate.

Time spent

The work spans several days with breaks. The saved step times of **[Test Crew 8]** add up to about 33 minutes. From its first to its last event it took 7 hours and 20 minutes, with idle gaps.

13. A NEW THREE-AGENT CARD

On 1 November I approved one new run of the **[card crew]**. The **[Art Director]** started three separate **[Codex]** agents: **[Card Data]**, **[Rule Audit]** and **[Print Match]**. Each returned six rows. Each role ran once, with no retry, no paid service and no new game.

Role	Input and result	Note
[Card Data], Sol Low	Read one fixed source packet. Returned six card rows with the exact printed text, the fields and the open items.	The right format did not mean right data. **[validate.py]** rejected wrong source labels and effect tags.
[Rule Audit], Sol High	Read its own fixed sources and the new rows. Flagged four cards; passed two.	A flag is a finding for review, not a confirmed bug. The missing value on **[Knockout!]** needs my ruling.
[Print Match], Sol High	Saw only six fixed card images, IDs and names. Read five; marked **[Mental Push]** as not readable.	It saw no data or audit text. It checked screen images, not the paper cards.

Result: stopped before release

The run stopped for my review, and no **[DT_Cards.csv]** was released. **[validate.py]** checked all three answers without a crash and kept the first print read unchanged. The flags, the wrong tags and labels, and the unreadable card blocked the release. No source was changed to get a pass. **[Test Crew 8]** is still the playable game output: a private prototype that I can show on request.

What the check caught

[Double Strike] says +4 in its text, but its data value says 3. **[Salvage Resources]** names the R.F.D. pile in its text, while a data field names discard. **[Mental Push]** has open reward symbols and mixes round and turn. Each finding names its source and waits for my ruling. The crew kept the printed text and did not guess a fix.

Controls and limits

Each agent was told to read only its own source packet or the six images. This is an instruction, not a locked sandbox. The token count of each agent was not available. Each open finding is an **[unknown]** with an **[agent decision-agent name]**, waiting for my ruling.

14. LEGEND

Each colored term in brackets links to its row in this Legend. Each row says what the term is, and links to the place that defines it. Links to the studio repository and the studio pages open only for people I give access to.

The color system

The three game loops keep their flat GDD colors, and a mix of two loops takes the GDD mixed color: Combat plus Dungeon Crawl is purple. Every other kind of term gets its own hue, chosen for what it means, and never a GDD loop hue. Inside a kind, the parent term is the most saturated and the darkest (never darker than 70%). Its children are lighter and a little softer, because they matter less, and each one has its own shade. So the hue tells the kind, the shade tells the rank, and the Legend row tells the exact thing.

Color	Kind	Means	Parent, then children
[Combat]	**Game loop**	The fight loop (GDD red).	-
[Dungeon Crawl]	**Game loop**	The dungeon loop (GDD blue).	-
[Landscape]	**Game loop**	The world map loop (GDD green).	-
[dungeon combat]	**Mixed loop**	Combat plus Dungeon Crawl (GDD purple).	-
[User]	**Me**	Gold: the king of the studio. Every order starts here.	-
[Lead Designer]	**Claude leads**	Warm red to orange, the colors closest to my gold, because they take my orders.	Claude and the Lead Designer, then the other leads
[Studio Director]	**Codex directors**	Cool cyan, the counterweight to the warm Claude leads.	Codex and the Studio Director, then the Art Director
[RAG files]	**Sources of truth**	What the agents read and write down. Deep royal indigo.	RAG files, then the artifact, the GDD, the rulebook, then the data files
[test crews]	**Crews**	Magenta, the teams that compete for my grade.	test crews, then Test Crews 1 to 9
[checks]	**Crew steps**	Green, the agents and checks that make the work grow.	checks, then each agent step (Card Data, Game Data and the others)
[Studio]	**Studio systems**	Metallic steel blue, the machines that run on their own.	Studio, then hooks and scripts
[unknown]	**Unknown**	Dark purple: a mystery not yet solved. The one shade allowed past 70% dark.	-
[_limbo]	**Trash**	Brown: the files nobody wants.	-

Terms

Term	Meaning	Where it is defined
[Combat]	The fight loop: card piles, movement, attack, defense, enemy turns, victory and defeat. It is the goal of this course.	GDD | Section 2
[dungeon combat]	The Combat loop inside a dungeon room. All test builds play this: one boss fight on a dungeon grid.	GDD | Section 2
[Dungeon Crawl]	The loop of moving through dungeon tiles and encounters that lead into combat. It comes next, if time allows.	GDD
[Landscape]	The loop of travel and events between places on the world map. It comes after the Dungeon Crawl.	GDD
[User]	Me, Piero, the studio owner. I set the goals and make the final choices. Only I can clear a chat or delete a file. The sketch calls me the Owner.	Section 3
[Claude]	Anthropic's AI model. Each Claude chat holds one studio role: Lead Designer, Studio Manager, Lead Engineer or Lead Web Designer.	Claude | Section 3
[Lead Designer]	Claude role. Designs the game and the crews, sends tasks and joins the results. It builds nothing itself.	Section 3
[Studio Manager]	Claude role. Its scripts watch the agents: work, usage, mailboxes and handoffs. It reports and alerts; it does not stop work itself.	Section 3 | Dev Log
[Lead Engineer]	Claude role. Builds and runs the environment, the engine, the scripts and its own helper agents.	Section 3
[Lead Web Designer]	Claude role. Builds the artifact and checks every fact on it and in every published output. A wrong fact goes back to the agent who logged it and its lead.	Section 3 | Dev Log
[Codex]	OpenAI's coding agent. Each Codex chat holds one director role: Studio Director or Art Director.	Codex | Section 3
[Studio Director]	Codex role. Runs the studio and speaks for me when I am away, for a set time. Its way of making choices is the decision rule for every agent.	Section 3 | Studio Home
[Art Director]	Codex role. Leads the look and the visual language of the game, and runs independent checks.	Section 3
[RAG files]	The approved documents each agent reads before it answers: the GDD, the rulebook, the data CSV, the art PDFs and the artifact. With me, they are the sources of truth. They have no search index yet.	GDD | Section 3
[F.A.M-E.D.]	Fire Age Main-Eterra Dungeoneers. My co-op dungeon and deck-building game for 1 to 4 players, first made as a tabletop game.	Website | GDD
[artifact]	The studio history log: the Dev Log and Studio Home pages on claude.ai. It is part of the RAG, read backwards: every lead and subagent logs each step on it, and a fact on it that disagrees with another source goes to a source of truth or to me.	Dev Log | Studio Home | Section 10

Term	Meaning	Where it is defined
[GDD]	The Game Design Document of F.A.M-E.D. It defines the loops, the rules and this color system.	GDD Draft 1
[rulebook]	The tabletop rules of F.A.M-E.D. When the rulebook and the website disagree, the website wins.	Website
[Dev Log]	The page of the artifact that lists every task with its date, time and author. Each agent writes in its own section, as it works.	Dev Log
[Studio Home]	The front page of the artifact: links, current work, decisions and open questions.	Studio Home
[card data]	The card rows that the Card Data agent made: one row of fields per card, beside the printed text.	combat-data.json
[rule audit]	The report that the Rule Audit agent made: each rule it checked and each open decision.	audit.json
[DT_Cards.csv]	The card table file the game engine loads, one row per card. Nothing goes in it until each card is confirmed.	Card row contract | Section 2
[records]	The saved inputs, models, call IDs and times of each agent call. They let anyone check what an agent was asked and what it gave back.	Crew trace | Section 6
[Knockout!]	Skill card SK1_01: your attacks deal double damage and stun this round. Used in 4 test crews.	Section 2
[Double Strike]	Skill card SK1_02: gain 1 extra attack roll this round. Used in all 9 test crews.	Section 2
[Mental Push]	Skill card SK1_03: draw 3 cards and shuffle 2 of them back. Its print was not readable in the card run.	Section 2
[Salvage Resources]	Skill card SK1_04: shuffle any 3 cards from your R.F.D. pile back into your deck.	Section 2
[Stand as One]	Skill card SK3_01: each party member can defend 1 attack per round with +2, for 1 round. Used in all 9 test crews.	Section 2
[Edan's Leap]	Skill card SK4_01: move up to 3 straight steps and ignore area effects and obstacles; in combat, land next to an enemy to attack with double damage.	Section 2
[agent crew]	A team of AI agents for one task: who does each step, and the order in which they pass work to each other.	Section 1 | Crew code
[test crews]	The nine crews, numbered 1 to 9 in build order, that each built the same combat demo from their own chart in competition for my grade. The Legend row of each crew names its chart.	Crew test plan | Section 7 | Studio Home
[card crew]	The six-card crew: Card Data, Rule Audit and Print Match. The Art Director started them as three Codex agents; validate.py checked their work.	Crew code | Section 13
[Test Crew 1]	Test crew number 1 in build order, built from chart 2 (Sequential baseline: one step after the other). Claude built it in web.	Folder | Chart 2
[Test Crew 2]	Test crew number 2 in build order, built from chart 3 (chart 3, adapted by Codex). Codex built it in web (canvas).	Folder | Chart 3
[Test Crew 3]	Test crew number 3 in build order, built from chart 4 (Joined data and code). Claude built it in Godot.	Folder | Chart 4
[Test Crew 4]	Test crew number 4 in build order, built from chart 5 (Separate build lane). Claude built it in web (Phaser).	Folder | Chart 5
[Test Crew 5]	Test crew number 5 in build order, built from chart 6 (chart 6, adapted by Codex). Codex built it in web (canvas).	Folder | Chart 6
[Test Crew 6]	Test crew number 6 in build order, built from chart 7 (Contract and expected cases). Claude built it in web (HTML and SVG).	Folder | Chart 7
[Test Crew 7]	Test crew number 7 in build order, built from chart 9 (Proof chain). Claude built it in web (Three.js, 3D).	Folder | Chart 9
[Test Crew 8]	Test crew number 8 in build order, built from chart 10 (Format first). Claude built it in web (Phaser). The winner: it puts a format check before every other check.	Folder | Chart 10 | Section 4
[Test Crew 9]	The last test crew: a best-of build made after the other eight, from a chain with sealed checks and separate lanes.	Folder | Section 7
[best playable]	My top play-test score, 9 of 10, shared by Test Crew 8 and Test Crew 6. Test Crew 8 won on interaction and animation.	Section 8
[checks]	All checks of a crew: the format check, the Rule Audit, the Print Match and the review.	Section 6
[Card Data]	Agent that reads the approved sources and writes one data row per card.	Agent file | Section 4
[Game Data]	The planned studio step that grows Card Data into all game data: the cards and every other game table.	Section 4
[Rule Audit]	Agent that checks each card row against the rules and lists the open decisions.	Agent file | Section 4
[Print Match]	Agent that sees only the card images, IDs and names, and compares each printed card with its data row.	Agent file | Section 4
[Game Builder]	Agent that joins the accepted data, the audit and my rulings into the playable build.	Section 4 | Folder
[format check]	A script that checks that each data row has the right shape before any agent reads it. It costs no model tokens.	validate.py | Section 4
[other checks]	The checks after the format check: the Rule Audit, the Print Match and the final review.	Section 4
[Studio]	Khi-Rho Studios: my roles, agents, mailboxes, scripts and pages that build F.A.M-E.D.	Section 3 | Studio Home
[decision rule]	How every agent decides: answer from the RAG files first, ask me when there is a gap, label each choice made without me, and never turn an unknown into a fact.	Section 3
[agent decision-agent name]	The tag an agent writes on each choice it makes while I am away: "agent decision" plus its role name. I use it to find and review those choices later.	Studio Home | Section 3
[Hook]	A script that runs on its own at a set event, such as the end of a reply, with no model call.	Section 3
[mailbox]	A folder for each role. Tasks, questions and results arrive there as dated files.	Mailboxes
[Watcher]	tools/inbox-watch.sh. Checks each role mailbox every 60 seconds and starts the Budget Guard every 5 minutes.	inbox-watch.sh
[Budget Guard]	tools/claude-budget/budget_guard.py. Reads the saved usage meter and alerts the chats. It does not stop work by itself.	budget_guard.py | Section 11
[Controller]	tools/studio-manager/controller.py. The Studio Manager status script: old work, mailbox coverage, watcher state, budgets and rule drift. It reports only.	controller.py | Section 11
[run_crew.py]	The card crew runner. It packs the sources, checks each answer, stops at 10 calls or 400,000 tokens, and holds back cards it cannot confirm.	run_crew.py | Section 5
[validate.py]	The script that checks the shape and the source labels of every card crew answer.	validate.py
[chart]	A Mermaid flow chart (a diagram written as text) of one crew: its agents, its links and its checks. Each test crew has the number of its chart.	Crew charts | Card crew chart
[crew-diagram.mmd]	The card crew chart as a Mermaid file (.mmd: a diagram written as plain text). GitHub reads the text and draws the chart.	crew-diagram.mmd
[README]	The front page of the public crew code: the game, the output, the crew roles, the folder map and what the files can show.	README.md
[repository]	The public GitHub repository of the crew code (famed-crew-share): agents, checks, runner and this report.	Public repository

Term	Meaning	Where it is defined
[studio repository]	The private GitHub repository of the whole studio: every file, log and handoff, with its history.	Studio repository
[handoff]	A dated file a chat writes before a clear: task state, open questions, file paths, costs, mistakes and running scripts.	Section 10
[Claude Code]	The app where the Claude chats and their helper agents run. Its Agent tool starts each agent.	Claude Code
[TTS]	Tabletop Simulator: the online tabletop where I first built and played F.A.M-E.D.	Tabletop Simulator
[Component Studio]	The card layout tool where I made the cards. Its CSV file gives each card its ID.	Component Studio
[Godot]	A free game engine. Test Crew 3 built its demo in it.	Godot
[Unreal Engine]	The game engine for the final F.A.M-E.D. build. No crew output is imported into it yet.	Unreal Engine
[gameplay feedback]	My written notes after I played each build. The Studio Director turned them into scores.	Section 8 | Feedback record
[CUP]	Commit the work, update the pages, then I push. It closes each task.	Section 10
[/clear]	Resets the memory of a chat. Only after a handoff file, and only when I choose it.	Section 10
[unknown]	A fact that no source of truth confirms. It stays open and flagged until a source or I settle it.	Section 6
[_limbo]	The trash folder: unwanted files go there. Nothing is deleted.	Section 10